

Les variables dans Thymeleaf

1. Qu'est-ce qu'une variable dans Thymeleaf ?

Vous connaissez déjà le concept de variable en Java. Thymeleaf a lui aussi ses variables, qui alimentent les expressions `${...}` d'un template.

On distingue deux familles :

- les **variables du contexte** (*context variables*), fournies par le contrôleur : les attributs de l'objet `org.springframework.ui.Model` ou de la requête `HttpServletRequest` ;
- les **variables locales**, définies dans le template lui-même (`th:each`, `th:with`), dont la portée est limitée à l'élément qui les déclare.

Les variables du contexte sont disponibles partout dans le template.

2. Variables du contexte

Côté contrôleur

@Controller

```
public class VariableController {  
  
    @GetMapping("/variable-example1")  
    public String variableExample1(Model model, HttpServletRequest request) {  
        // variable1 : attribut du Model  
        model.addAttribute("variable1", "Valeur de variable1 !");  
  
        // variable2 : attribut de la requête  
        request.setAttribute("variable2", "Valeur de variable2 !");  
  
        return "variable-example1"; // templates/variable-example1.html  
    }  
}
```

Rappel : depuis Spring Boot 3, `HttpServletRequest` provient du paquet `jakarta.servlet.http` (et non plus `javax`). En pratique, on privilégie le `Model` ; l'utilisation directe de la requête reste possible mais rarement nécessaire.

Côté template : variable-example1.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Variables</title>
</head>
<body>
  <h1>Variables</h1>

  <!-- Texte brut : L'expression n'est PAS évaluée -->
  <h4>${variable1}</h4>

  <!-- Évaluée par Thymeleaf -->
  <span th:text="${variable1}">valeur par défaut</span>

  <!-- Syntaxe en ligne (inlining) : évaluée aussi -->
  <p>Variable 2 = [[${variable2}]]</p>
</body>
</html>
```

Trois points à retenir :

- Un `${variable1}` écrit directement dans le contenu d'une balise est affiché tel quel : Thymeleaf ne traite que les attributs `th:*` et les expressions en ligne `[[...]]`.
- Il faut utiliser `th:text` plutôt que `th:utext`, car il échappe le HTML (protection contre les failles XSS). `th:utext` n'est à utiliser que pour un contenu de confiance.
- `[[${...}]]` est l'équivalent « en ligne » de `th:text` (et `[(${...})]` de `th:utext`).

3. Variables locales

Les variables locales sont définies dans le template. Elles n'existent que dans la portion du template où elles sont déclarées.

Variables créées par `th:each`

Dans l'exemple suivant, `flower` et `iter` n'existent que dans la boucle qui les déclare.

```
@GetMapping("/variable-in-loop")
public String variableInLoop(Model model) {
    String[] flowers = {"Rose", "Lily", "Tulip", "Carnation", "Hyacinth"};
    model.addAttribute("flowers", flowers);
    return "variable-in-loop";
}
```

variable-in-loop.html :

```
<h1>Variables locales : flower, iter</h1>
<table border="1">
```

```

<tr>
  <th>N°</th>
  <th>Nom de la fleur</th>
</tr>
<!-- flower : élément courant ; iter : état de l'itération -->
<tr th:each="flower, iter : ${flowers}">
  <td th:text="${iter.count}">N°</td>
  <td th:text="${flower}">Nom de la fleur</td>
</tr>
</table>

```

La deuxième variable (iter) est l'**objet d'état de l'itération**. Il expose les propriétés suivantes :

Propriété	Signification
index	Position de l'élément, à partir de 0
count	Position de l'élément, à partir de 1
size	Nombre total d'éléments
current	Élément courant
even / odd	Position paire ou impaire
first / last	Premier ou dernier élément

Si l'on ne nomme pas cette variable, Thymeleaf la crée automatiquement sous le nom de la variable d'itération suivi de Stat (ici flowerStat).

Variables créées par th:with

L'attribut th:with définit une ou plusieurs variables locales, avec une syntaxe d'affectation classique.

```

@GetMapping("/variable-example3")
public String variableExample3(Model model) {
    String[] flowers = {"Rose", "Lily", "Tulip", "Carnation", "Hyacinth"};
    model.addAttribute("flowers", flowers);
    return "variable-example3";
}

```

variable-example3.html :

```

<h1>th:with</h1>

<!-- Variable locale : flower0 -->
<div th:with="flower0 = ${flowers[0]}">
  <h4>[[${flower0}]]</h4>
</div>

<!-- Variables locales : flower1, flower2 -->
<div th:with="flower1 = ${flowers[1]}, flower2 = ${flowers[2]}">

```

```

    <h4>[[${flower1}]], [[${flower2}]]</h4>
</div>
<hr>

<!-- Variables Locales : firstName, lastName, fullName -->
<div th:with="firstName = 'James', lastName = 'Smith',
            fullName = |${firstName} ${lastName}|">
    Prénom : <span th:text="${firstName}"></span><br>
    Nom : <span th:text="${lastName}"></span><br>
    Nom complet : <span th:text="${fullName}"></span>
</div>

```

Remarques :

- Les définitions sont évaluées de gauche à droite : `fullName` peut donc utiliser `firstName` et `lastName`, déclarées juste avant dans le même `th:with`.
- La syntaxe `|...|` (*literal substitution*) remplace avantageusement la concaténation `${firstName} + ' ' + ${lastName}`.
- **Ordre d'exécution** : lorsque plusieurs attributs `th:*` figurent sur la même balise, `th:each` et `th:if` sont traités avant `th:with`. Une variable définie par `th:with` n'est donc pas visible dans le `th:each` ou le `th:if` de la même balise.
- Pour regrouper plusieurs éléments sans ajouter de balise HTML, on utilise `<th:block>`.

Créer une liste directement dans le template

```

<h1>th:with (liste)</h1>

<th:block th:with="flowers = ${ {'Rose', 'Lily', 'Tulip'} }">
    <table border="1">
        <tr>
            <th>N°</th>
            <th>Fleur</th>
        </tr>
        <tr th:each="flower, state : ${flowers}">
            <td th:text="${state.count}">N°</td>
            <td th:text="${flower}">Fleur</td>
        </tr>
    </table>
</th:block>

```

Avec SpringEL, l'expression `{ 'Rose', 'Lily', 'Tulip' }` crée une `java.util.List` (non modifiable), et non un tableau Java. Cette technique est pratique pour des maquettes ou de petites listes de constantes. Pour des données réelles, le contrôleur reste le bon endroit pour les préparer.

4. Autres sources de variables

Outre le `Model` et la requête, Thymeleaf met à disposition :

- `${param.nom}` : paramètre de la requête HTTP (par exemple `?nom=Tom`) ;
- `${session.cle}` : attribut de la session HTTP ;
- `${application.cle}` : attribut du contexte de l'application ;
- `${@monBean.methode()}` : appel d'une méthode d'un bean Spring.

Depuis Thymeleaf 3.1, l'accès direct aux objets `#request`, `#session` et `#servletContext` n'est plus autorisé dans les expressions, pour des raisons de sécurité. La bonne pratique consiste à transmettre explicitement au `Model` les données dont la vue a besoin.

Enfin, `th:object` définit un **objet sélectionné** auquel on accède avec la syntaxe `*{...}` :

```
<div th:object="${person}">
  <span th:text="*{firstName}">Prénom</span>
  <span th:text="*{lastName}">Nom</span>
</div>
```

Résumé

- Les attributs du `Model` (ou de la requête) sont des variables disponibles dans tout le template.
- `th:each` et `th:with` créent des variables locales, limitées à l'élément qui les déclare.
- L'objet d'état d'itération donne accès à `index`, `count`, `size`, `first`, `last`, etc.
- `th:text` et `[[...]]` échappent le contenu ; `th:utext` et `[(...)]` ne le font pas.
- Dans une même balise, `th:each` et `th:if` sont évalués avant `th:with`.

Exercices proposés

1. Afficher une liste de fleurs avec une ligne sur deux colorée, en utilisant `iter.odd` et `th:classappend`.
2. Afficher « Dernier élément » à côté de la dernière fleur grâce à `iter.last`.
3. Définir avec `th:with` une variable `total` égale à la somme `prix * quantite` d'une ligne de commande, puis l'afficher.
4. Lire le paramètre `?nom=...` de l'URL et afficher un message de bienvenue avec `${param.nom}`.
5. Expliquer pourquoi `<div th:with="liste = ${flowers}" th:each="f : ${liste}">` ne fonctionne pas comme on pourrait l'attendre, et proposer une correction (indice : ordre d'exécution des attributs, `<th:block>`).